

Project 4: Personal Expense Tracker

SkillInfy Tech Internship Project Implementation Guide

Project Description: Design a simple tracker to record daily expenses and generate reports. It stores date-wise expenses in a file or database and allows filtering and report generation.

1. Goals & Skills Learned

- **Data Persistence:** Utilizing the built-in `json` file storage architecture to save structured objects across system sessions securely.
- **Date/Time Handling:** Implementing native `datetime` module tracking to accurately timestamp ledger lines.
- **Analytical Grouping:** Computing sum totals and dynamically filtering transactional logs based on structured category matches.
- **Modular Programming:** Segregating structural mutation tasks (adding data) away from presentation tasks (generating analysis summaries).

2. System Architecture & Flow

The program maintains a file-backed JSON transactional record. It uses a clean command-line selection loop where inputs are evaluated against data structures to calculate historical spending metrics on-demand.

3. Complete Source Code Implementation

```

import json
import os
from datetime import datetime

# Persistent storage configuration
DATA_FILE = "expense_records.json"

def initialize_ledger():
    """Ensures file ledger structure exists safely on disk without overwriting data."""
    if not os.path.exists(DATA_FILE):
        with open(DATA_FILE, "w") as file:
            json.dump([], file)

def load_expenses():
    """Loads the list of recorded transaction entries from storage."""
    initialize_ledger()
    try:
        with open(DATA_FILE, "r") as file:
            return json.load(file)
    except json.JSONDecodeError:
        return []

def save_expenses(expenses):
    """Saves the entire list state back to the backing JSON structure."""
    with open(DATA_FILE, "w") as file:
        json.dump(expenses, file, indent=4)

def log_expense():
    print("\n--- Log Direct Expense Item ---")
    expenses = load_expenses()

    try:
        amount = float(input("Enter Amount Spent (e.g., 450.50): ").strip())
        if amount <= 0:
            print("❌ Error: Spending amount must be greater than zero.")
            return
    except ValueError:
        print("❌ Error: Invalid numeric input provided.")
        return

    category = input("Enter Category (e.g., Food, Travel, Rent, Utilities): ").strip()
    if not category:
        category = "Miscellaneous"

    description = input("Enter Brief Description: ").strip()

    # Capture timestamp natively in standard YYYY-MM-DD formatting
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M")

    # Package tracking map structure
    new_record = {
        "timestamp": timestamp,
        "amount": amount,

```

```

        "category": category,
        "description": description if description else "N/A"
    }

    expenses.append(new_record)
    save_expenses(expenses)
    print(f"✅ Logged entry of {amount} under '{category}' successfully.")

def view_all_expenses():
    print("\n--- Historical Expense Ledger ---")
    expenses = load_expenses()

    if not expenses:
        print("Your transaction history is completely empty.")
        return

    print(f"{'Date & Time':<18} | {'Category':<15} | {'Amount':<10} | {'Description'}")
    print("-" * 70)

    grand_total = 0.0
    for item in expenses:
        print(f"{'item['timestamp']':<18} | {'item['category']':<15} | {'item['amount']':<10.2f} | {'item['description']':<25}")
        grand_total += item["amount"]

    print("-" * 70)
    print(f"{'GRAND TOTAL RECORDED SPENDING':<36} | {grand_total:<10.2f} | ")
    print("-" * 70)

def generate_category_report():
    print("\n--- Expense Breakdown Analysis Report ---")
    expenses = load_expenses()

    if not expenses:
        print("No transactional records found to compile analytical results.")
        return

    # Aggregate transaction statistics inside grouping dictionary
    breakdown = {}
    total_spent = 0.0

    for item in expenses:
        cat = item["category"]
        amt = item["amount"]
        breakdown[cat] = breakdown.get(cat, 0.0) + amt
        total_spent += amt

    print(f"{'Category Classification':<25} | {'Total Accumulated Outflow':<30} | {'Percentage':<10}")
    print("-" * 75)
    for cat, amt in breakdown.items():
        percentage = (amt / total_spent) * 100
        print(f"{'cat':<25} | {'amt':<30.2f} | {'percentage: .1f}%")
    print("-" * 75)
    print(f"{'Combined Outflow Summary':<25} | {total_spent:<30.2f} | 100.0%")

```

```

def run_tracker():
    while True:
        print("\n=====")
        print("      PERSONAL EXPENSE TRACKER      ")
        print("=====")
        print("1. Log New Expense Entry")
        print("2. Display Ledger & Grand Total")
        print("3. Generate Analytical Category Report")
        print("4. Close Tracker Application")
        choice = input("Select an action (1-4): ").strip()
        if choice == "1":
            log_expense()
        elif choice == "2":
            view_all_expenses()
        elif choice == "3":
            generate_category_report()
        elif choice == "4":
            print("\nSafely closing tracking systems. Good luck budgeting!")
            break
        else:
            print("❌ Invalid selection. Please specify options 1 through 4.")
    if __name__ == "__main__":
        try:
            run_tracker()
        except KeyboardInterrupt:
            print("\n\nSession closed prematurely via user override action.")

```

4. Verification & Testing Steps

1. Initiate terminal execution context: `python expense_tracker.py`
2. Input mock records via Option 1 across distinct categories like *Food* and *Utilities*. Confirm the generation of the file `expense_records.json` on disk.
3. Trigger Option 2 to confirm calculations compute aggregate totals correctly.
4. Run Option 3 to test percentage tracking math maps categories to accurate percentage proportions.